

5ypqcjdkw

March 28, 2026

```
[1]: !pip install monai pynrrd scikit-learn joblib -q
```

```
2.7/2.7 MB
32.3 MB/s eta 0:00:000:0100:01
```

```
[2]: import os, torch, numpy as np
from tqdm import tqdm
import matplotlib.pyplot as plt
import joblib

from monai.transforms import *
from monai.data import Dataset, DataLoader
from monai.networks.nets import SwinUNETR, SegResNet
from monai.inferers import sliding_window_inference

from sklearn.cluster import KMeans
from sklearn.model_selection import KFold, StratifiedKFold, cross_val_score, \
    cross_val_predict
from sklearn.ensemble import GradientBoostingRegressor, \
    GradientBoostingClassifier
from sklearn.metrics import *

DEVICE = torch.device("cuda")
```

```
<frozen importlib._bootstrap_external>:1301: FutureWarning: The cuda.cudart
module is deprecated and will be removed in a future release, please switch to
use the cuda.bindings.runtime module instead.
2026-03-28 11:18:35.269083: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1774696715.470718      55 cuda_dnn.cc:8579] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1774696715.532383      55 cuda_blas.cc:1407] Unable to register
```

cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has already been registered

W0000 00:00:1774696716.026555 55 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.

W0000 00:00:1774696716.026601 55 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.

W0000 00:00:1774696716.026604 55 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.

W0000 00:00:1774696716.026606 55 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.

```
[3]: # DATA
image_dir = '/kaggle/input/datasets/srujan215/sinus-dataset/NasalSeg/images'
label_dir = '/kaggle/input/datasets/srujan215/sinus-dataset/NasalSeg/labels'

files=[]
for f in sorted(os.listdir(image_dir)):
    if f.endswith('_img.nrrd'):
        img=os.path.join(image_dir,f)
        lbl=os.path.join(label_dir,f.replace('_img.nrrd','_seg.nrrd'))
        if os.path.exists(lbl):
            files.append({'image':img,'label':lbl})

print("Total:", len(files))
```

Total: 130

```
[4]: # TRANSFORMS
trans = Compose([
    LoadImaged(keys=['image','label']),
    EnsureChannelFirstd(keys=['image','label']),
    ScaleIntensityRanged(keys=['image'], a_min=-1000, a_max=400, b_min=0,
↪b_max=1, clip=True),
    Resized(keys=['image'], spatial_size=(96,96,64)),
    Resized(keys=['label'], spatial_size=(96,96,64), mode='nearest'),
    ToTensord(keys=['image','label'])
])

loader = DataLoader(Dataset(files, trans), batch_size=1)
```

```
[5]: # MODELS
swin = SwinUNETR( in_channels=1, out_channels=6, feature_size=48).to(DEVICE)
seg = SegResNet(in_channels=1, out_channels=6, init_filters=32).to(DEVICE)
```

```

swin.load_state_dict(torch.load("/kaggle/input/models/srujan215/
    ↪sinus-segmentation/pytorch/default/1/best_swin.pth", map_location=DEVICE,
    ↪weights_only=True))
seg.load_state_dict(torch.load("/kaggle/input/models/srujan215/
    ↪sinus-segmentation/pytorch/default/1/best_seg.pth", map_location=DEVICE,
    ↪weights_only=True))

swin.eval(); seg.eval()

```

```

[5]: SegResNet(
  (act_mod): ReLU(inplace=True)
  (convInit): Convolution(
    (conv): Conv3d(1, 32, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1,
1, 1), bias=False)
  )
  (down_layers): ModuleList(
    (0): Sequential(
      (0): Identity()
      (1): ResBlock(
        (norm1): GroupNorm(8, 32, eps=1e-05, affine=True)
        (norm2): GroupNorm(8, 32, eps=1e-05, affine=True)
        (act): ReLU(inplace=True)
        (conv1): Convolution(
          (conv): Conv3d(32, 32, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
        )
        (conv2): Convolution(
          (conv): Conv3d(32, 32, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
        )
      )
    )
    (1): Sequential(
      (0): Convolution(
        (conv): Conv3d(32, 64, kernel_size=(3, 3, 3), stride=(2, 2, 2),
padding=(1, 1, 1), bias=False)
      )
      (1): ResBlock(
        (norm1): GroupNorm(8, 64, eps=1e-05, affine=True)
        (norm2): GroupNorm(8, 64, eps=1e-05, affine=True)
        (act): ReLU(inplace=True)
        (conv1): Convolution(
          (conv): Conv3d(64, 64, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
        )
        (conv2): Convolution(

```

```

        (conv): Conv3d(64, 64, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
    )
)
(2): ResBlock(
  (norm1): GroupNorm(8, 64, eps=1e-05, affine=True)
  (norm2): GroupNorm(8, 64, eps=1e-05, affine=True)
  (act): ReLU(inplace=True)
  (conv1): Convolution(
    (conv): Conv3d(64, 64, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
  )
  (conv2): Convolution(
    (conv): Conv3d(64, 64, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
  )
)
(2): Sequential(
  (0): Convolution(
    (conv): Conv3d(64, 128, kernel_size=(3, 3, 3), stride=(2, 2, 2),
padding=(1, 1, 1), bias=False)
  )
  (1): ResBlock(
    (norm1): GroupNorm(8, 128, eps=1e-05, affine=True)
    (norm2): GroupNorm(8, 128, eps=1e-05, affine=True)
    (act): ReLU(inplace=True)
    (conv1): Convolution(
      (conv): Conv3d(128, 128, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
    )
    (conv2): Convolution(
      (conv): Conv3d(128, 128, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
    )
  )
  (2): ResBlock(
    (norm1): GroupNorm(8, 128, eps=1e-05, affine=True)
    (norm2): GroupNorm(8, 128, eps=1e-05, affine=True)
    (act): ReLU(inplace=True)
    (conv1): Convolution(
      (conv): Conv3d(128, 128, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
    )
    (conv2): Convolution(
      (conv): Conv3d(128, 128, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
    )
  )
)

```

```

    )
    )
)
(3): Sequential(
  (0): Convolution(
    (conv): Conv3d(128, 256, kernel_size=(3, 3, 3), stride=(2, 2, 2),
padding=(1, 1, 1), bias=False)
  )
  (1): ResBlock(
    (norm1): GroupNorm(8, 256, eps=1e-05, affine=True)
    (norm2): GroupNorm(8, 256, eps=1e-05, affine=True)
    (act): ReLU(inplace=True)
    (conv1): Convolution(
      (conv): Conv3d(256, 256, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
    )
    (conv2): Convolution(
      (conv): Conv3d(256, 256, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
    )
  )
  (2): ResBlock(
    (norm1): GroupNorm(8, 256, eps=1e-05, affine=True)
    (norm2): GroupNorm(8, 256, eps=1e-05, affine=True)
    (act): ReLU(inplace=True)
    (conv1): Convolution(
      (conv): Conv3d(256, 256, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
    )
    (conv2): Convolution(
      (conv): Conv3d(256, 256, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
    )
  )
  (3): ResBlock(
    (norm1): GroupNorm(8, 256, eps=1e-05, affine=True)
    (norm2): GroupNorm(8, 256, eps=1e-05, affine=True)
    (act): ReLU(inplace=True)
    (conv1): Convolution(
      (conv): Conv3d(256, 256, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
    )
    (conv2): Convolution(
      (conv): Conv3d(256, 256, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
    )
  )
)

```

```

(4): ResBlock(
  (norm1): GroupNorm(8, 256, eps=1e-05, affine=True)
  (norm2): GroupNorm(8, 256, eps=1e-05, affine=True)
  (act): ReLU(inplace=True)
  (conv1): Convolution(
    (conv): Conv3d(256, 256, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
  )
  (conv2): Convolution(
    (conv): Conv3d(256, 256, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
  )
)
)
)
(up_layers): ModuleList(
  (0): Sequential(
    (0): ResBlock(
      (norm1): GroupNorm(8, 128, eps=1e-05, affine=True)
      (norm2): GroupNorm(8, 128, eps=1e-05, affine=True)
      (act): ReLU(inplace=True)
      (conv1): Convolution(
        (conv): Conv3d(128, 128, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
      )
      (conv2): Convolution(
        (conv): Conv3d(128, 128, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
      )
    )
  )
  (1): Sequential(
    (0): ResBlock(
      (norm1): GroupNorm(8, 64, eps=1e-05, affine=True)
      (norm2): GroupNorm(8, 64, eps=1e-05, affine=True)
      (act): ReLU(inplace=True)
      (conv1): Convolution(
        (conv): Conv3d(64, 64, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
      )
      (conv2): Convolution(
        (conv): Conv3d(64, 64, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
      )
    )
  )
  (2): Sequential(

```

```

(0): ResBlock(
  (norm1): GroupNorm(8, 32, eps=1e-05, affine=True)
  (norm2): GroupNorm(8, 32, eps=1e-05, affine=True)
  (act): ReLU(inplace=True)
  (conv1): Convolution(
    (conv): Conv3d(32, 32, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
  )
  (conv2): Convolution(
    (conv): Conv3d(32, 32, kernel_size=(3, 3, 3), stride=(1, 1, 1),
padding=(1, 1, 1), bias=False)
  )
)
)
)
(up_samples): ModuleList(
  (0): Sequential(
    (0): Convolution(
      (conv): Conv3d(256, 128, kernel_size=(1, 1, 1), stride=(1, 1, 1),
bias=False)
    )
    (1): UpSample(
      (upsample_non_trainable): Upsample(scale_factor=(2.0, 2.0, 2.0),
mode='trilinear')
    )
  )
  (1): Sequential(
    (0): Convolution(
      (conv): Conv3d(128, 64, kernel_size=(1, 1, 1), stride=(1, 1, 1),
bias=False)
    )
    (1): UpSample(
      (upsample_non_trainable): Upsample(scale_factor=(2.0, 2.0, 2.0),
mode='trilinear')
    )
  )
  (2): Sequential(
    (0): Convolution(
      (conv): Conv3d(64, 32, kernel_size=(1, 1, 1), stride=(1, 1, 1),
bias=False)
    )
    (1): UpSample(
      (upsample_non_trainable): Upsample(scale_factor=(2.0, 2.0, 2.0),
mode='trilinear')
    )
  )
)
)

```

```

(conv_final): Sequential(
  (0): GroupNorm(8, 32, eps=1e-05, affine=True)
  (1): ReLU(inplace=True)
  (2): Convolution(
    (conv): Conv3d(32, 6, kernel_size=(1, 1, 1), stride=(1, 1, 1))
  )
)
)
)

```

```

[6]: # FEATURE FUNCTION
def extract_features(mask):
    mask = mask.cpu().numpy()
    total = (mask>0).sum()
    ratio = total / mask.size
    per_class = [(mask==c).sum() for c in range(1,6)]
    coords = np.argwhere(mask>0)
    if len(coords)>0:
        dims = coords.max(0) - coords.min(0) + 1
        bbox = np.prod(dims)
        extent = total / bbox if bbox>0 else 0
        centroid = coords.mean(0)
    else:
        dims=[0,0,0]; bbox=extent=0; centroid=[0,0,0]
    return [total, ratio, bbox, extent, *dims, *centroid, *per_class]

```

```

[7]: # FEATURE EXTRACTION
X,y=[],[]

with torch.no_grad():
    for b in tqdm(loader):
        img=b['image'].to(DEVICE)
        lbl=b['label']

        p1=sliding_window_inference(img,(128,128,64),2,swin)
        p2=sliding_window_inference(img,(128,128,64),2,seg)

        pred=(p1+p2)/2
        mask=torch.argmax(pred,dim=1)[0]

        X.append(extract_features(mask))
        y.append((lbl>0).sum().item())

X=np.array(X)
y=np.array(y)

```

0%| | 0/130 [00:00<?, ?it/s]/usr/local/lib/python3.12/dist-packages/monai/inferers/utils.py:226: UserWarning: Using a non-tuple sequence

for multidimensional indexing is deprecated and will be changed in pytorch 2.9; use `x[tuple(seq)]` instead of `x[seq]`. In pytorch 2.9 this will be interpreted as tensor index, `x[torch.tensor(seq)]`, which will result either in an error or a different result (Triggered internally at `/pytorch/torch/csrc/autograd/python_variable_indexing.cpp:347.`)

```
win_data = torch.cat([inputs[win_slice] for win_slice in
unravel_slice]).to(sw_device)
/usr/local/lib/python3.12/dist-packages/monai/inferers/utils.py:370:
UserWarning: Using a non-tuple sequence for multidimensional indexing is
deprecated and will be changed in pytorch 2.9; use x[tuple(seq)] instead of
x[seq]. In pytorch 2.9 this will be interpreted as tensor index,
x[torch.tensor(seq)], which will result either in an error or a different result
(Triggered internally at
/pytorch/torch/csrc/autograd/python_variable_indexing.cpp:347.)
out[idx_zm] += p
100%|          | 130/130 [02:01<00:00, 1.07it/s]
```

```
[8]: kmeans = KMeans(n_clusters=4, random_state=42)
y_cls = kmeans.fit_predict(y.reshape(-1,1))

print("Class distribution:", np.bincount(y_cls))
```

Class distribution: [42 19 26 43]

```
[9]: # MODELS
reg = GradientBoostingRegressor(n_estimators=300, learning_rate=0.01,
    ↪max_depth=5)
clf = GradientBoostingClassifier(n_estimators=200, learning_rate=0.05,
    ↪max_depth=3, subsample=0.8, min_samples_leaf=5)
```

```
[14]: # CV

# Regression
cv_reg = KFold(n_splits=5, shuffle=True, random_state=42)
r2 = cross_val_score(reg,X,y,cv=cv_reg,scoring='r2')

# Classification
cv_cls = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
acc = cross_val_score(clf,X,y_cls,cv=cv_cls,scoring='accuracy')

ypr = cross_val_predict(reg,X,y,cv=5)
yprc = cross_val_predict(clf,X,y_cls,cv=5)

print("MAE:", mean_absolute_error(y,ypr))
print("RMSE:", np.sqrt(mean_squared_error(y,ypr)))

print("F1:", f1_score(y_cls,yprc,average='weighted'))
```

```
print(classification_report(y_cls,ypc))

print("R2:", r2.mean(), "+-", r2.std())
print("Accuracy:", acc.mean(), "+-", acc.std())
```

MAE: 1865.1849333369128

RMSE: 2475.7670398786913

F1: 0.8215791615791616

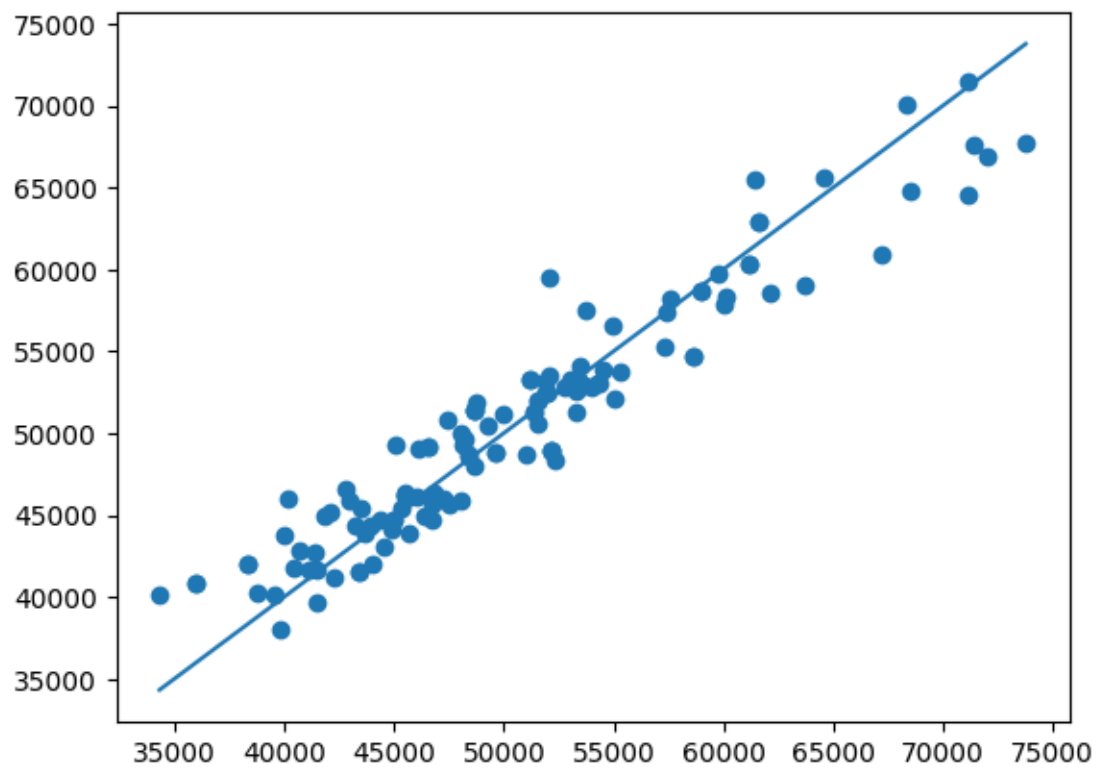
	precision	recall	f1-score	support
0	0.79	0.74	0.77	42
1	0.89	0.89	0.89	19
2	0.83	0.77	0.80	26
3	0.81	0.91	0.86	43
accuracy			0.82	130
macro avg	0.83	0.83	0.83	130
weighted avg	0.82	0.82	0.82	130

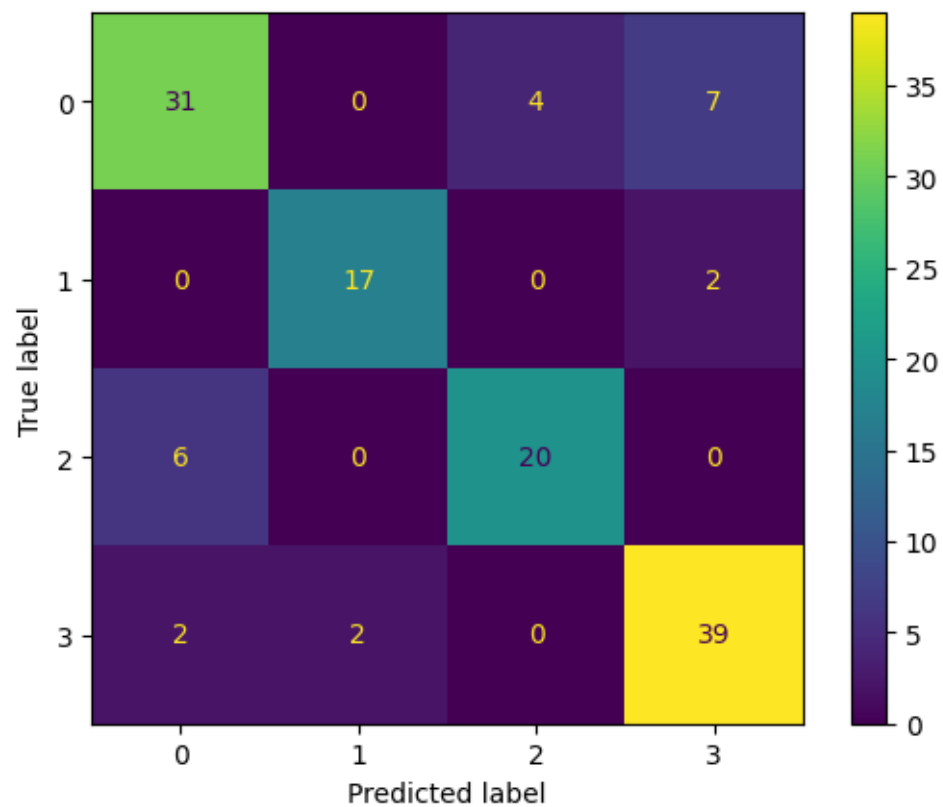
R2: 0.9334538636928873 +- 0.021144135791729825

Accuracy: 0.8307692307692307 +- 0.052171769100963605

```
[15]: plt.scatter(y,ypr)
plt.plot([y.min(),y.max()], [y.min(),y.max()])
plt.show()

ConfusionMatrixDisplay(confusion_matrix(y_cls,ypc)).plot()
plt.show()
```





```
[16]: joblib.dump(reg, 'reg.pkl')
      joblib.dump(clf, 'clf.pkl')
      np.save('X.npy', X)
      np.save('y.npy', y)
```